Project Note: 12-Month Price Forecasting

AI/ML Developer assignment for Way2Agribusiness India Pvt. Ltd. · Shriyam · +91 97980 44462 · shriyamgopalgupta@gmail.com

The task

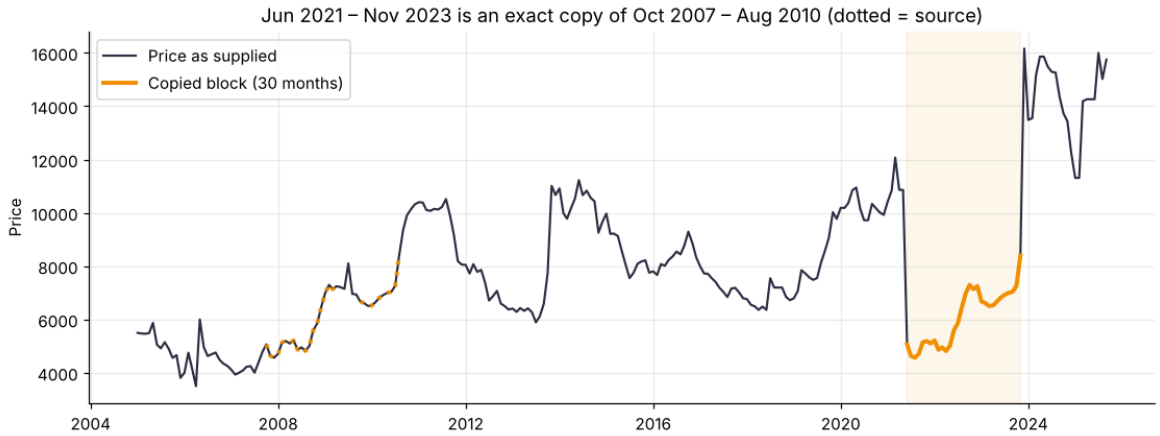
Forecast the average monthly price for the next 12 months from 249 months of history (Jan 2005 – Sep 2025), evaluate the model on both training and future months, and explain how to act on the forecast and run it in production with Django, FastAPI, Next.js and React Native.

What I did and how

Step	How	Outcome
1. Data checks	Checked the calendar for gaps, nulls, duplicates and invalid prices; searched the series against every shifted copy of itself for repeated stretches; flagged one-off spikes (more than 22% away from the 7-month rolling median); listed repeated values.	No explicit missing values, but hidden problems found (below).
2. Cleaning	Set the bad months to missing and re-estimated them in log space: remove the month-of-year effect, interpolate the level, add the effect back. Kept real market jumps.	32 months repaired, 9 stale months flagged.
3. Exploration	Trend/seasonal decomposition (STL), stationarity tests (ADF, KPSS), autocorrelation of monthly returns.	Strong trend, weak seasonality, price close to a random walk.
4. Modelling	Compared 10 model families (naive baselines, ETS, Theta, ARIMA/SARIMA, LightGBM) and grid-searched 71 SARIMA settings. All models work on log prices.	Selected **ARIMA(1,1,0)** on log prices.
5. Evaluation	13 rolling 12-month backtests (2014–2020) for selection, then a final check on the last 12 months (Oct 2024 – Sep 2025), which no model saw during selection. Metrics: MAPE, RMSE, MASE, bias, interval coverage.	8.7% error on the unseen year; all 12 months inside the 95% range.
6. Forecast	Refit on all data; forecast Oct 2025 – Sep 2026 with 80% and 95% ranges.	About 15,800, roughly flat, with a wide range.
7. Deployment	Built a working Django REST Framework app and a FastAPI service around the model, with 13 automated tests.	APIs reproduce the notebook forecast exactly.

Key finding: a hidden copy-paste error in the data

Jun 2021 – Nov 2023 (30 months) is an exact, value-for-value copy of Oct 2007 – Aug 2010. It made the price look like it crashed 53% in one month and then jumped 92% when the copy ended. Treating these months as missing matters: leaving them in made the price look about 70% more volatile and hid a real momentum effect. Apr–May 2006 (3,500 then 6,000) was also a data-entry spike and was repaired the same way.



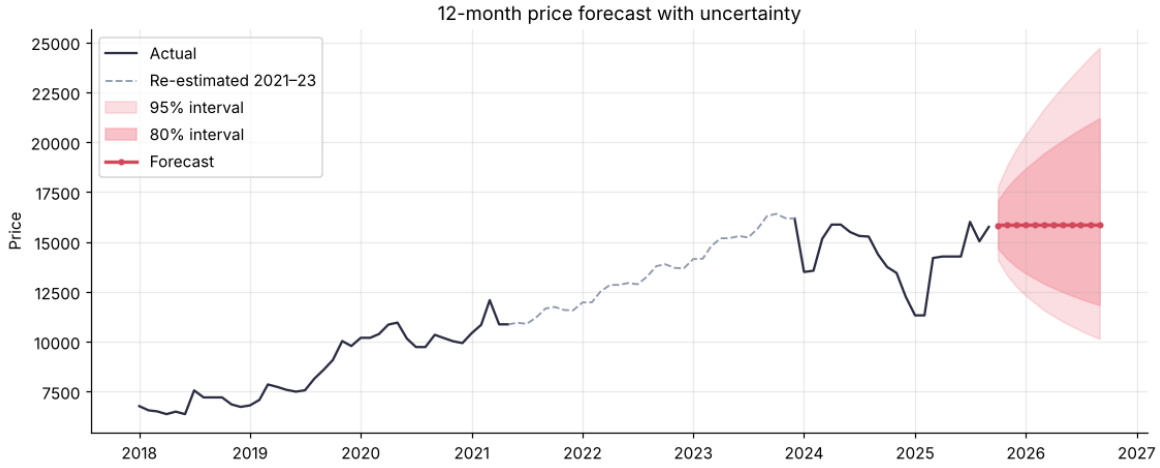
Orange: the copied months (solid) and the 2007–2010 data they were copied from (dotted).

Why ARIMA(1,1,0)?

- **In plain words:** next month's % change = about 10% of this month's change + an unpredictable shock. Monthly shocks average about ±6%, so the forecast quickly settles flat and the uncertainty range widens.
- **Most accurate on the unseen year** (lowest error of all 10 candidates) and in the top group on the backtests.
- **Simple and robust:** one parameter, fits in milliseconds, easy to explain and monitor.
- **Honest uncertainty:** its 95% ranges contained 99% of backtest months and all 12 holdout months.
- **LightGBM did worst on the unseen year (16% error):** with about 250 data points and only past prices, it learned false patterns. It would become useful once external drivers are added (mandi arrivals, rainfall, stocks, policy).

Evaluation	Average error (MAPE)	vs seasonal-naive (MASE)	Inside 95% range
Training (1 month ahead)	4.2%	–	–
Unseen year, Oct 2024 – Sep 2025	8.7%	0.71	12 of 12 months
13 rolling backtests (12 months ahead)	10.5%	0.55	99%

The forecast: Oct 2025 – Sep 2026



Central forecast about 15,800, roughly flat. By Sep 2026: 80% chance between about 11,800 and 21,200; 95% chance between about 10,100 and 24,700. Because the price is mostly shocks, the business value is in planning around the range: hedging part of purchases, staggered buying and selling, warehouse storage, and alerts when the price leaves the 80% band.

How it runs in production

- **Django + DRF (system of record):** stores monthly prices with data-quality flags, keeps a registry of model versions, and runs a monthly *retrain_forecast* job (clean → fit → quality gate → promote). A new model goes live only if it beats the random-walk benchmark and has honest intervals; old versions stay for rollback.
- **FastAPI (real-time predictions):** */predict* returns the forecast with ranges and accepts newer prices to update the forecast instantly without retraining. Django calls */reload* to swap in a new model with no downtime.
- **Next.js and React Native** read the same versioned JSON API. Mobile caches the last forecast for poor connectivity and gets push alerts when the price crosses the forecast range.
- **Monitoring:** monthly accuracy, bias, interval coverage and volatility drift, with alerts that trigger retraining.

Files: full notebook *Price_Forecasting_Shriyam.ipynb*. Code and tests: github.com/Shriyam-22/price-forecasting-django-fastapi